

Lavoisier: Model-Based Systems Engineering in a Type System

An evidence-first account of modelling real-world systems in stable Rust, with conservation enforced by the compiler

Tony Walmsley

6 October 2026 — version 0.1

Contents

Abstract	3
Document status	3
1 Motivation and hypothesis	3
2 The method: an evidence loop	4
3 The approach: R1–R19 by theme	5
3.1 Conservation and ownership	5
3.2 Quantities, units and capacity	5
3.3 States and characteristics	6
3.4 Requirements and traceability	6
3.5 The boundary	7
3.6 Variability, people and money	7
3.7 The verification gate	7
4 Implementation	8
4.1 model-core	8
4.2 The kernel macros	8
4.3 The two models	8
4.4 The CI gate	9
4.5 The spec-to-model workflow, and CS-1 as the worked account	9
5 The experiments	10
5.1 EXP-04: where conservation checking actually lives	11
5.2 EXP-03: the affine gap, quantified	12
5.3 EXP-08: the boundary holds, under discipline	12
5.4 EXP-10..12: the open questions close	12
6 Findings and results	12
6.1 What the compiler genuinely guarantees	13
6.2 The post-monomorphization boundary	13
6.3 The layered regime for silent loss	13
6.4 The SIGBUS, and errors as a designed surface	13

6.5	Measured costs	14
6.6	Negative results that shaped the design	14
7	Limitations	14
8	Related work	15
9	Future work	16
9.1	The planned deliverables (PLAN steps 8–11)	16
9.2	The case-study ladder	16
9.3	Open questions	17
10	Conclusions	17
	Appendix A: Glossary	18
	Appendix B: The rules R1–R19, one line each	19
	Appendix C: Findings index by theme	20

Abstract

Lavoisier is an experiment in using a statically-typed programming language — stable Rust, with no external dependencies beyond a test harness — as a medium for model-based systems engineering. Resources, requirements and processing states are types; processes are functions that take everything they need by value and return everything they produce; and a model that double-books a technician, loses mass between steps, overdraws a budget or uses a part no earlier step produced does not compile. The project was run evidence-first: nineteen written requirements of the modelling approach itself (R1–R19), twelve isolated experiments with pass/fail criteria (EXP-01..EXP-12), a consolidated findings log of fifty-four entries (F-001..F-054), a reusable library (`model-core`), two complete downstream models, and a six-step verification gate. This paper gives the full account: the method, the rules and the evidence behind them, what the compiler genuinely guarantees, what needs a layered regime of lints, tripwires and tests, the measured costs, and the limitations stated plainly.

Document status

Version	0.1 (first public draft)
Date	2026-10-06
Covers	R1–R19; EXP-01..EXP-12; F-001..F-054; <code>model-core</code> ; the pilot workshop model; case study CS-1
Status	Living document

Versioning convention. This is a living document, extended rather than rewritten. Each future case study (CS-2..CS-5, see §9) and each major deliverable (generated diagrams, process document generation, the DSL, model analysis) adds sections and appends findings; existing sections are corrected but not restructured. Minor versions (0.2, 0.3, ...) mark those extensions; the document stays at 0.x until the case-study ladder is complete and the open questions of §9.3 are resolved or formally parked. Every factual claim is verifiable against the repository: rule numbers (R), finding numbers (F) and experiment numbers (EXP) are cited inline and indexed in the appendices.

1 Motivation and hypothesis

Engineering plans and specifications are mostly written in prose, and prose hides contradictions. A technician booked on two jobs at once, a step that consumes a part no earlier step produced, material that silently disappears between operations, waste with no disposal route, a budget quietly overspent — none of these is visible on the page, because nothing on the page is obliged to balance. The errors surface later — in procurement, on the shop floor, at integration — where they are expensive.

The project’s founding observation is that this class of error is *bookkeeping* error, and bookkeeping can be audited mechanically. The name honours Antoine Lavoisier — “*Rien ne se perd, rien ne se crée, tout se transforme*”: nothing is lost, nothing is created, everything is transformed. If an engineering plan is written so that every resource — material, energy, time, money — must balance across every step, then an auditor that checks the books on every change will catch the contradictions at design time.

The specific hypothesis is that a modern type system can be that auditor. Rust was chosen because its ownership rules give the central property away: values move. A value passed to a function is gone from the caller; it cannot be used twice, and (with `Clone` and `Copy` forbidden on resources) it cannot be duplicated. If every *thing* in a plan is a type, and every *work step* is a function whose inputs and outputs must balance, then an impossible plan is a type error — and the compiler, software’s strictest proof-reader, rejects it with a message saying what is wrong and where. The project deliberately does not build software in the ordinary sense; it borrows software’s auditor.

Two refinements make this an experiment rather than a manifesto. First, the hypothesis was *expected to fail somewhere*: the stated aim (instructions.md, Summary) is “to find out how far a type system can be pushed for model-based systems engineering, and to record the limitations and complications that turn up along the way”. Where there was a choice, the project preferred the approach that puts more checking into the compiler, *even if the types get complicated*, precisely to find where that breaks down. Second, the claims are confined to consistency, not truth: the method proves a plan internally coherent — it does not prove the plan matches reality (a kettle might not really need 550,000 J), and it does not simulate performance. Validation against reality is an open question (§9.3), kept deliberately distinct from the verification this paper is about.

The constraint set matters to the result: **stable Rust only** (no nightly features such as `generic_const_exprs`), no dependencies except the `trybuild` test harness as a dev-dependency, and the project’s own units and type-level-number machinery rather than crates like `uom` or `typenum` (R8, R12). Every measurement in this paper was taken on `rustc 1.98.1` (stable, September 2026) on Apple Silicon macOS.

2 The method: an evidence loop

The project’s second experiment — alongside the technical one — was methodological: every design rule had to be backed by evidence before it was adopted, and every loss had to be recorded rather than papered over. The loop ran as follows.

1. **Requirements agreed in writing.** The requirements *of the modelling approach itself* live in `instructions.md` as numbered rules R1–R19 (distinct from the REQ–NNN requirements of any modelled system). Changes to them are agreed between author and implementer before being committed; nothing is reinterpreted silently.
2. **Isolated experiments.** Each risky claim the rules make became one self-contained experiment crate (EXP-01..EXP-12), with build steps and pass/fail criteria fixed in advance; the first nine ran in parallel with no shared code. Each produced a `RESULTS.md` with a per-criterion verdict (**pass** / **pass with complications** / **fail**), verbatim compiler output judged for how a modeller would read it, candidate findings, and an adopt/adapt/reject recommendation.
3. **A findings log.** A single consolidation pass turned the candidate findings into `FINDINGS.md` (R14): stable IDs F-001 onwards, each recording what could not be expressed (or only with complications), what it cost, and the workaround adopted, with evidence citations into the experiment crates. Later findings were added by the library and model builds themselves; a finding that is later resolved gains a dated `RESOLVED` note (the F-036 pattern) rather than being rewritten.
4. **Rules updated.** The experiment results were written back into R1–R19 as amendments — sometimes strengthening a rule (Peano numbers adopted for capacities, EXP-01), sometimes weakening or re-scoping one (the `Supplier` trait declared discrete-only, F-028), and three times creating a new rule outright (R17–R19 were open questions until

EXP-10..12 answered them).

5. **Implementation.** Only then was the real library built: `model-core`, plus a pilot model (`pilot-workshop`) that exercises every feature, gated end to end by a CI script (§4.4).
6. **Case studies.** New models are now written from natural-language specifications via `SPEC_TEMPLATE.md`, each chosen to stress something no earlier model did; CS-1 (making a pot of tea) completed the first full spec-to-model round trip (§4.5).

Evidence-first mattered for three reasons. It kept the rules honest — every convention in §3 cites the finding that forced it, and several intuitive designs died in experiments before reaching the library (binary type-level numbers, F-012; the type-selected outcome token, F-042; the aggregate “workshop” parameter, F-024). It made the *negative* results durable: the findings log preserves what failed and why, with verbatim compiler output, so a future revisit does not re-run dead ends. And it surfaced genuinely unexpected results that casual development would likely have misattributed — most dramatically a deterministic compiler crash (F-034, §6.4) and the discovery that the compiler’s own suggested fixes systematically recommend conservation violations (F-007, §5.2).

3 The approach: R1–R19 by theme

The nineteen rules are stated normatively in `instructions.md`; this section synthesizes them thematically.

3.1 Conservation and ownership

The core rule, R1, is Lavoisier’s principle operationalized: a process function transforms inputs into outputs with no side effects — it may neither create resources from nothing nor consume an input without turning it into some output (product, by-product or waste). Everything a process needs is moved in by value; everything it produces is returned. R2 sharpens this into strict exclusivity: resources are never borrowed or shared (no `&T`, `&mut T`, `Rc` or `Arc` in process signatures); a reusable resource — a person, a tool, a location — is moved in and *returned*. The compiler consequently enforces one resource in one process at a time (F-025), and a double-booked person fails with an error pointing at both competing uses.

“No creation from nothing” is made structural rather than conventional: resources have private constructors (sealed types with at least one private field), so only the boundary code that represents real-world arrival may mint them (R1, R12). Changing an amount is itself a process (R3), with the balance checked by a compile-time assertion (`const { assert!(IN == A + B) }`) — splitting 2000 g into 1500 g + 600 g is a compile error.

The known gap is stated in the rules themselves: Rust’s types are affine, not linear, so a value can be *dropped* silently. “Nothing is lost” is therefore only partly a compile-time property; the remainder is covered by a layered regime (R1, quantified in §6.3): `#[must_use]` on every resource with the deny-lints that back it, a panicking “tripwire” destructor on every consumable resource that converts silent test-time leaks into named failures, a clippy configuration banning the escape hatches, and per-branch test coverage. The rules are explicit that the set is only sound as a whole, and that the residual gaps are accepted and documented rather than denied.

3.2 Quantities, units and capacity

Every quantity and unit is a type that also carries its value (R7): integers in base units — grams, millimetres, milliseconds, square and cubic millimetres, millikelvin, milliamperes,

joules, and minor currency units — with no floating point. Unit mixing is a type error: grams cannot meet millimetres, and (by R19) pence cannot meet euro cents. The project writes its own units library (R8) rather than depending on existing crates (§8).

Discrete items are separate objects, never counts (R13): a bolt is a value, and a box of bolts *contains* its bolt values. Capacity is type-level (R12): the box’s remaining contents are a type-level list whose length is the count — a box of 100 bolts is literally a type one hundred levels deep, on purpose — so an exhausted supplier simply does not implement `Supplier`, and drawing a fifth bolt from a box typed as holding four is a compile error. The type-level numbers behind this are the project’s own Peano naturals, adopted over a binary encoding by experiment (EXP-01, F-012), with measured guidance: capacities to ~100 are free, ~500 is the comfort ceiling, and every participating crate sets `#![recursion_limit = "2048"]` (F-010).

Continuous material takes the opposite representation (R15): a sealed container with a const-generic magnitude (`GasBottle<5000>`, `Battery<E>`, `Account<BALANCE>`), drawn down by an explicit draw process in which the caller states the remainder and a const assert checks the arithmetic. Overdrawing is then a conservation violation with *no compiling remainder* — finite capacity falls out of conservation for free. Rates and wall-clock time stay outside the model (R9, R15): what is conserved is the amount — joules, grams, person-milliseconds — not watts or schedules. Time as a costed input is a budget on the resource that spends it (`Person<300_000>`), drawn down like the gas bottle.

3.3 States and characteristics

A thing’s characteristics are encoded in types (R6), two ways with fixed roles settled by experiment (EXP-05): catalogues of orthogonal values are parameterized types (`Bolt<Size, Material, Length>`, each slot policed by a “kind” trait so transposed arguments fail at the construction site, F-018), while requirements and capabilities are marker traits, bridged from the catalogue by one-line blanket impls (F-019). Each processing state is its own type (R9): a `Plate`, a `DrilledPlate` and a `ScrapPlate` are three types, converted only by processes. This is what makes “product never produced” a compile error — “expected `DrilledPlate`, found `Plate`” reads directly as *these plates have not been drilled yet* (F-023) — and the same rule extends to failure states (a broken drill bit is not a drill bit) and safety states (a fitted guard is not a loose guard, R18).

R9 also fixes what the model is *about*: connections, not sequences. Process signatures declare which outputs feed which inputs; any order satisfying the data dependencies type-checks, and independent branches may run concurrently because strict conservation proves they share no resource. The model deliberately contains no clock.

3.4 Requirements and traceability

Requirements of the modelled system are first-class artifacts (R10): each is a named trait whose documentation begins with its id (`REQ-NNN`), composed of characteristic supertraits with a blanket impl, and used as a bound on every process that demands it. Because the blanket impl makes satisfaction invisible to text search (F-020), each satisfying type carries a `/// Satisfies: REQ-NNN doc tag backed by a compile-checked assertion` — the tag is for grep, the assertion is for truth, a stale tag a compile error naming the missing characteristic. Tests are tagged `/// Verifies: REQ-NNN` (R5 requires at least one test per process, and per `Result` arm for fallible processes). A dependency-free shell script, `trace.sh`, turns the source into a traceability report — requirement → defined at / satisfied by / verified by — and fails CI on any warning, including a requirement with no verifying test. The discipline that keeps grep

reliable (exact tag case, tags only in `///` doc comments, bounds on the `fn-name` line, no tags inside macro invocations) is itself linted by the same script (F-021, F-037).

3.5 The boundary

Everything enters the model through explicit suppliers and leaves through explicit consumers (R12), which are themselves resources, passed by value, with finite, type-level capacity — and which are *strict*: a supplier takes nothing but itself and yields one item plus its next state; anything that needs payment or paperwork is a process chain, not a supplier. Repeated use goes through a recursive helper trait (`SupplyN<N>`), so “take four bolts” costs one where-clause (F-014). Continuous sources are draw-style boundary processes, not `Supplier` impls — the trait is provably discrete-only on stable Rust (F-028).

Practically unbounded environments — the atmosphere, mains water, the grid — are `Next = Self` boundary objects, legal *only* at the boundary (inside the model they would mint or swallow resources, exactly what R1 forbids) and always placeholders. The placeholder convention (R12) is the model’s named ignorance: a greppable `/// Placeholder:` tag on every assumed supplier, sink, outcome source and exchange rate, each a designed refinement point. Consumed time and other events worth recording go to **History** (R16), a sealed boundary consumer that defuses what it consumes but keeps a value-level record — “the past” as an explicit sink, doubling as the execution record. Concurrent branches carry separate histories merged at joins into an honest partial order: no interleaving is invented.

3.6 Variability, people and money

The last three rules were open questions answered by the final experiment batch. **Fallible processes** (R17, from EXP-10): a process that can fail returns `Result<OkBundle, FailBundle>` where *both* arms conserve the same inputs — a scrapped part is a product, not a disappearance — with one `const assert` per branch, both checked at every call site. Variability enters only at the boundary as a sealed, runtime-valued outcome token that flows cannot read; the type-selected alternative was built and rejected because it collapses fallibility back into two infallible processes (F-042). Outcome bundles deliberately implement no `Debug`, which makes `.unwrap().expect()` — the panicking shortcut past the failure arm — a compile error (F-047). Rework is bounded by provisioning: a retry consumes provisioned reserves, so unbounded retry is inexpressible, which is the honest statement of real rework (F-050).

Qualifications and safety (R18, from EXP-11): certifications are marker traits on a sealed `Qualified` wrapper around the common `Person` — never on `Person` itself, where a single impl would certify everyone in the model at once (F-043) — and safety states follow one-type-per-state, so an unqualified operator and an unfitted guard are distinct, REQ-phrased compile errors. **Money** (R19, from EXP-12): each currency is its own dimension in integer minor units; cash and accounts reuse the R15 container machinery unchanged, so overspending is the standard overdraft error; vendors implement `Consumer` only at their exact price, which moves wrong payments to type-check time with the correct price named in the error (F-051); and currency exchange — value-equivalence at a stated integer rate, not conservation — is confined to the boundary, `const-asserted` exactly, so silent rounding is unrepresentable (F-052).

3.7 The verification gate

R4 and R5 close the loop: compile errors must expose model mismatches, with pinned compile-fail regressions, and unit tests must verify every transformation. The crucial caveat

— the project’s single most consequential finding (F-001, §5.1) — is that conservation asserts fire at monomorphization, during code generation, so `cargo check` and the editor diagnostics built on it cannot see them. The gate (§5.4) is therefore built on full builds, never on `cargo check` alone.

4 Implementation

4.1 model-core

The library (`model/model-core`) is deliberately small — eight public modules and a feature-gated fixtures module:

- `nat` — Peano type-level naturals with generated aliases `N0..N1000` (the alias table is written by a shipped generator script, because `stable macro_rules!` cannot synthesize identifiers, F-013);
- `list` — type-level lists of real values, whose length *is* the count;
- `quantity` — the `Unit` kind trait, the eight base units, sealed `Qty<V, U>`, and conserving `split/combine`;
- `resource` — the sealing/tripwire kernel (§4.2);
- `boundary` — `Supplier` (discrete-only), `Consumer`, recursive `SupplyN/ConsumeList`, and the generic access processes, all carrying modeller-phrased `#[diagnostic::on_unimplemented]` messages;
- `common` — `Person` with an R15 time budget, the `Qualified` wrapper, `Organisation`, `Location`, `Labour` and its production-legal `TimeLedger` sink (F-035);
- `history` — the R16 execution history;
- `requirement` — the R10 pattern as macros compatible with `trace.sh`.

`model-core` is infrastructure: it defines no REQ-NNN requirements itself. Its crate documentation carries the conservation obligations every modelling crate must adopt (recursion limit, `forbid(unsafe_code)`, `deny-lints`, tripwires) and a condensed error-reading guide — because the sealing pattern is “one attribute away from broken” with no compiler warning (F-005), the rules are stated where every modeller will read them.

4.2 The kernel macros

The hard-won part of the library is boilerplate generation. Sealing a resource correctly takes about ten lines of tripwire machinery plus the sealing checklist per type (F-008); the `resource` module’s macros (`consumable_resource!`, `container_resource!`, `reusable_resource!`, `draw_process!`, `outcome_token!` and friends) generate the sealed struct, the `#[must_use]` attribute, the tripwire `Drop`, the sanctioned defuse site, boundary constructors and test-support fixtures together, so a downstream resource is one declaration. The macros initially accepted only non-generic types (F-036); extending them to generic parameters and payload fields required a token-munching parser with documented grammar limits, and surfaced its own finding (F-040: tripwired wrappers must hold *untripwired* contents, or defusing the wrapper would erase the contents’ leak protection). EXP-12’s verdict is the pay-off measured: modelling money needed **zero** new library machinery — `container_resource!` and `draw_process!` fit cash and accounts verbatim (F-051).

4.3 The two models

pilot-workshop is the validation model, built alongside the library and exercising all of it: a steel sheet is cut into plates, drilled (a certified operator behind a fitted guard, drawing

time from a budget; drilling can fail, scrapping the plate or snapping the bit), and fastened into an assembly with four catalogue bolts from a BoltBox supplier; swarf goes to a contents-keeping bin with a sealed disposal path (F-039); labour is recorded into per-branch histories merged at the join; money pays for parts through an exact-price vendor. Five requirements (REQ-001..REQ-005) are defined, satisfied and verified through `trace.sh`.

cs1-pot-of-tea is the first *case-study* model — built not alongside the library but from an agreed natural-language specification (§4.5).

4.4 The CI gate

`model/ci.sh` is the whole verification story in one command, six steps, each existing because a finding proved it necessary:

1. **Full cargo build** — conservation E0080s fire at monomorphization; a gate resting on `cargo check` would pass violating models (F-001).
2. **cargo test** — unit and integration tests, `trybuild` compile-fail cases, tripwire `should_panic` demonstrations, and the `rustdoc compile_fail` doc-tests that are the only automated regressions for conservation violations (`trybuild` cannot see them, F-003).
3. **Clippy under -D warnings** plus the explicit restriction lints — the conservation lint set is only sound as a whole (F-007).
4. **A plain no-features build** of every member's production targets — during `cargo test`, feature unification compiles production sources against the test-support-featured library, so only this build proves production code cannot reach fixture constructors (F-004).
5. **trace.sh** — the traceability report; any warning (including an unverified requirement) exits nonzero.
6. **A feature-placement audit** — a `grep` proving test-support never appears under any [dependencies]-like section.

The model workspace currently carries 87 test functions plus the pinned compile-fail and doc-test regressions — over a hundred automated checks run end to end on every change.

4.5 The spec-to-model workflow, and CS-1 as the worked account

`SPEC_TEMPLATE.md` defines how a model begins: a natural-language specification whose sections map mechanically onto model elements — §2 requirements become REQ-NNN traits verbatim, §3 resources become sealed types with their kinds (discrete/continuous/reusable/product), states and quantities, §4 the boundary table (every waste in §5 must have a destination row), §5 processes with their balance lines, §6 flows as connections with the concurrency stated, §7 assumptions destined for Placeholder: tags. The template's governing rule: *anything the specification leaves out does not silently default — it comes back as a numbered question*.

CS-1, making a pot of tea, ran this loop in full. The spec (v0.2 agreed, v0.3 as implemented) models one person, an electric kettle, a teapot and a box of 40 teabags inside a kitchen boundary: five processes (fill, load, boil, pour-and-brew, empty the bin), with water (1500 g), energy (550,000 J drawn; 500,000 J embodied in the boiling water, 50,000 J kettle losses) and time (75,000 ms across four draws from a 300,000 ms budget) conserved per process — the brew's mass balance is $1500 + 9 = 1473 + 36$ — and one genuine ordering freedom: the kettle boils with *no person*, so loading the pot before or during the boil are both valid orders, and both compile as separate integration flows. Four requirements (boiling water only; exactly 3 teabags;

spent bags to the food-waste bin; waste heat accounted to the kitchen air) are enforced as REQ traits; the implementation produced one new positive pattern (F-054: a requirement-bounded process can consume a state-changing resource and keep REQ-phrased errors, via associated-const magnitudes and a permit-gated extraction).

The round trip was itself instrumented: implementation sent back eight numbered feedback items — among them, that the template should state where a time draw lives (adjacent process, F-048), should ask whether a process takes its consumers as parameters or the flow routes its outputs, and should distinguish *assert* from *structural* balance lines — all folded into the template at the review gate. Most significantly, the workflow found a real tooling flaw: CS-1’s spec numbered its requirements REQ-001..REQ-004, but `trace.sh`’s id namespace is workspace-global and the pilot already owned REQ-001..REQ-005 — implementing the spec literally left the CI gate **green** while the traceability report silently merged CS-1’s tags under the pilot’s requirements (F-053), the dangerous green-but-wrong direction. CS-1 shipped as REQ-006..REQ-009 with the mapping documented, and the template now instructs spec authors to allocate workspace-unique ids. That the first real use of the workflow surfaced a silent hole in the verification tooling — and closed it — is itself a result: the method audits its own instruments.

5 The experiments

Each experiment tested one risky claim in isolation, with pass/fail criteria fixed in advance. The table gives one row each; the subsections discuss the decisive ones.

EXP	What it tested	Verdict (headline)	Key fact
01	Type-level naturals at useful sizes (R12)	pass w/ complications	Peano adopted; <code>recursion_limit</code> $\approx N+3$; 1.0 s build at $N=100$, 4.6 s at $N=1000$; binary flat ~ 1 s but ~ 25 impls vs 7 and no less-than
02	Suppliers/consumers as type-level lists (R12)	pass	<code>SupplyN</code> makes N draws one where-clause; capacity 100 builds in ~ 2.6 s; 130 breaks the default recursion limit
03	How close to “nothing silently lost”? (R1)	fail at compile time; pass as layered regime	8 leak paths; 3 have no compile-time detection; tripwire converts 6 of 8 into test failures
04	Compile-time conservation of quantities (R3, R7)	pass w/ complications	Violations are genuine E0080s — but post-monomorphization: <code>cargo check</code> , <code>rust-analyzer</code> and <code>trybuild</code> all pass them

EXP	What it tested	Verdict (headline)	Key fact
05	Marker traits vs type parameters (R6)	pass	24-combination catalogue: 33 lines parameterized vs 121 hand-written markers; adding a value: 2 lines vs 6+
06	Mechanical traceability (R10)	pass	~150-line dependency-free <code>trace.sh</code> ; zero false positives/negatives over 11 edge cases
07	Flows, exclusivity, reusable threading (R2, R9)	pass	Double-booking errors point at both uses; a process that swallows a tool compiles at its own definition site
08	The privacy boundary across crates (R1)	pass w/ complications	No safe-Rust hole in sealed types; seven one-attribute-away holes catalogued; <code>cargo test</code> alone cannot prove the fixture boundary
09	Continuous resources, budgets, sinks (R15)	pass	The whole R15 combination composes; errors print decimal magnitudes; <code>Supplier</code> proved discrete-only
10	Fallible processes (candidate R17)	pass w/ complications	Both arms conserve, checked at every call; <code>.unwrap()</code> does not compile; cost: 8 const parameters on one process
11	Qualifications and safety (candidate R18)	pass w/ complications	REQ-phrased refusals (“this person may not drill...”); wrapper ~34 lines once vs ~40 duplicated lines per parallel person type
12	Money as a conserved dimension (candidate R19)	pass	Zero new library machinery; exact-price vendors make wrong payments <i>editor-visible</i> ; exchange is integer-exact

5.1 EXP-04: where conservation checking actually lives

The project’s central promise — “mass in = mass out, checked by the compiler” — is true, with an asterisk that shapes everything else. On stable Rust the check is an inline `const { assert!(A + B == IN) }` in a generic function; a violation is a genuine, unavoidable E0080 — but it evaluates only when a fully concrete instantiation is reached *during code generation*. Verified directly: compiling the violating program with `--emit=metadata` (what `cargo check`

does) succeeds; a full build fails. So editor diagnostics report violating models as fine; try-build, which runs `cargo check` internally, reports “Expected test case to fail to compile, but it succeeded”; and generic code never instantiated is never checked (F-001, F-003). The whole verification architecture — `ci.sh` gating on builds, conservation regressions as `rustdoc compile_fail` doc-tests, R5’s every-process-instantiated test discipline, and the documentation warning that *while you type, your editor is lying to you about balances* — follows from this one measurement.

5.2 EXP-03: the affine gap, quantified

EXP-03 catalogued eight ways a resource can silently die (end-of-scope drop, `let _ =`, shadowing, `mem::forget`, `mem::drop`, `struct-pattern .. drops`, early return, panic unwinding) and crossed them with every stable counter-mechanism. The matrix is the project’s honest boundary: three paths have no compile-time detection by any stable mechanism, and panic unwinding evades everything (a tripwire that stays armed during unwind double-panics and aborts the whole test binary — worse than the leak). The adopted regime — `must_use` + deny-lints at compile time, the tripwire Drop at test time, clippy’s banned-methods list, branch-coverage tests — converts six of the eight paths into failures. The experiment’s most striking single finding is F-007: each mechanism’s official fix-it suggestion is the *next leak path* (`unused_must_use` suggests `let _ =`; `let_underscore_drop` suggests `drop(...)`; for a swallowed parameter `rustc` suggests borrowing, which R2 forbids). A subset of the lints gives false safety, and modellers must be explicitly taught to ignore the compiler’s advice on this axis.

5.3 EXP-08: the boundary holds, under discipline

Sealed types showed no safe-Rust hole from downstream — but seven one-attribute slips silently reopen the boundary (`#[derive(Default)]`, `#[derive(Clone)]`, public fields, a pub unit struct, a pub enum, any public data-to-Self function, and `unsafe`), each demonstrated by a passing test from the user crate. The sealing rules exist because the compiler will not defend the pattern against its own author; each rule is mechanically greppable. The experiment also settled test-fixture mechanics: `#[cfg(test)]` helpers can never serve downstream tests, and the `test-support` feature alternative leaks into production sources during `cargo test` via feature unification — measured with a probe that `cargo test` accepted and plain `cargo build` rejected (F-004). Hence `ci.sh` steps 4 and 6.

5.4 EXP-10..12: the open questions close

The final batch ran against the finished `model-core` and converted the three open questions into R17–R19. Two results stand out. First, a negative with teeth: selecting a process’s outcome with *types* is fully expressible on stable Rust — and useless, because the outcome becomes part of the flow’s static text and nothing ever forces the unchosen arm to be handled; fallibility requires exactly one runtime-valued sealed token per process kind (F-042). Second, an unplanned positive: because outcome bundles hold sealed resources and derive no `Debug`, `Result::unwrap()` and `.expect()` — the universal panicking shortcuts — simply do not compile (E0277), closing at type-check time a hole the design had expected to police by convention (F-047).

6 Findings and results

Fifty-four findings condense into six lessons.

6.1 What the compiler genuinely guarantees

At type-check time — visible in editors, in `cargo check` and to `trybuild` — the compiler enforces: exclusivity (a resource in use by one process at a time, F-025); no duplication and no creation from nothing (sealed types, F-005/F-006); state correctness (a product used before the process that makes it, a failure state continuing a success flow — F-023, F-047); unit and currency segregation (E0308, F-051); requirement satisfaction (REQ-bounded processes, F-044); capacity on discrete suppliers and consumers (an empty box does not implement `Supplier`, F-015); transposition in catalogues (kind traits, F-018); and wrong-price payment (exact-price vendors, F-051). These are the hard wins: they need no tests and no coverage, only the sealing checklist.

6.2 The post-monomorphization boundary

Everything arithmetic — mass balances, energy balances, overdraw of containers and budgets, per-branch conservation of fallible processes, exchange-rate exactness — is a genuine compile error that fires *late*: at monomorphization, invisible to `cargo check`, editors and `trybuild` (F-001, F-003, F-027, F-045). The project treats this as a first-class boundary with its own architecture: CI gates on full builds; conservation regressions are `rustdoc compile_fail` doc-tests; every process must be instantiated by a test; and the error-reading guide teaches that the real call site of a cross-crate E0080 is in the “while instantiating `fn draw_gas::<6000, 0, 5000>`” note, not the primary span. One genuine consolation, measured in EXP-09: unlike Peano capacities, `const-generic` magnitudes print as *decimal numbers* in every diagnostic, so continuous-resource errors are the most readable in the project (F-027).

6.3 The layered regime for silent loss

“Nothing is silently lost” is a property of the regime, not the type system (F-002): compile-time protection for discard-by-statement, `let _`, `forget` and explicit drop; test-time protection (the tripwire) for used-then-dropped values, forgotten match-arm fields and abandoned end states (the empty gas bottle, the zero-change `Money<0>` — F-032); and two accepted residuals — untested branches (branch coverage *is* leak coverage, R5) and panic unwinding, under which a bundle of five resources was demonstrated to vanish with no abort and no report (F-047). Three structural corollaries surfaced only in real builds: whoever mints a tripwired type must also ship a production-legal consumer for it, or downstream flows literally cannot account for it (F-035); contents-keeping consumers need a sealed recursive disposal path, or emptying the bin fires every kept tripwire (F-039); and tripwired wrappers must hold untripwired contents (F-040).

6.4 The SIGBUS, and errors as a designed surface

The project found one way to kill the compiler outright: a contents-keeping consumer with no decreasing space parameter diverges trait resolution, which at the default recursion limit is a graceful E0275 — but at the project-mandated `recursion_limit = "2048"` is a deterministic `rustc` SIGBUS with no diagnostic at all (F-034). The shape is now a hard rule (every contents-keeping consumer has a decreasing space parameter; R16’s type-level-recording `History` variant is forbidden for the same reason), and “compiler dies with no error” has a documented first suspect.

More broadly, error-message quality was treated as a first-class engineering concern — the errors *are* the product’s user interface — and the findings record both directions. Curated: `#[diagnostic::on_unimplemented]` turns capacity and requirement failures into

sentences (“BoltBox<Nil> cannot supply anything: it is empty”; “this person may not drill: Person<5000> is not a certified drilling operator (REQ-004)”) — with the measured caveats that the attribute only fires on the trait-bound path (F-015), only the root obligation’s message is shown (so it must sit on the requirement trait, F-044), and the message is fixed per trait, not per impl. Uncurable and documented instead: aliases are erased (N41 vs N42 is two 40-deep Succ chains to count by hand), long types spill to side files from capacity ~130, duplicate bound errors appear at one call site (F-009), and composed processes emit echo E0080s whose notes point inside the library (F-051). The shipped error-reading guide, the deliberately Debug-less bundles and the “ignore the fix-its” rule are all consequences of taking this surface seriously.

6.5 Measured costs

The approach’s costs are quantified rather than waved at. **Compile time** is a non-issue at the intended scale: ~2.6 s for a full capacity-100 supplier cycle, ~1 s at Peano N=100, superlinear only past N≈500 (4.6 s at N=1000) — the guidance is capacities ≤ ~500. **Line counts**: the parameterized catalogue costs 33 lines where hand-written markers cost 121 at 24 combinations, and grows by 2–3 lines per new value; a sealed resource is one macro line (after F-036’s resolution); a hand-sealed wrapper like `Qualified` is ~34 lines once, versus ~40 duplicated lines *per type* on the rejected parallel-person design (F-043); `trace.sh` is ~150 lines of shell. **Const-parameter burden** is the real ergonomic tax: output magnitudes cannot be computed on stable, so callers state every total (F-022), budget parameters infect every signature their resource passes through with the running balance re-derived by hand (F-030), and a fallible process carries both branches’ splits — eight const parameters on the pilot’s drilling step, `drill_fallible::<1000, 4000, 5000, 450, 440, 10, 430, 20>` (F-045). None of this is a soundness cost — a wrong number anywhere is the E0080 — but it is typing, and it is the strongest argument for the planned DSL.

6.6 Negative results that shaped the design

Several rejections are as load-bearing as the adoptions: binary type-level numbers (scale beautifully, cost 3× the machinery, and the capacity lists are O(N)-deep anyway — F-012); aggregate resource bundles (they over-claim, deleting the concurrency R9 exists to allow — F-024); markers on `Person` (certifies everyone at once — F-043); type-selected outcomes (F-042); shadow lints (they fire on the conservation-correct idiom — F-007); and `Supplier` for continuous material (E0207 makes it impossible, and the packet workaround makes non-multiple amounts unreachable — F-028).

7 Limitations

The project’s credibility rests on stating these plainly; they are the documented record, not concessions extracted by critics.

- **Conservation errors are invisible while editing** (F-001, F-003). The balance checks fire only on a full build. `cargo check`, rust-analyzer’s default diagnostics and `trybuild` all pass violating models; generic code never instantiated is never checked. The mitigations (§4.4) are real but are mitigations: the editor is lying about balances until the build runs.
- **Silent loss is not fully closed** (F-002). The layered regime leaves two holes: a leak on a branch no test exercises is caught by nothing, and panic unwinding evades every layer silently. A panicking model run has already failed, but its accounting is void.

- **The boundary is discipline, not construction** (F-005). Sealing is one attribute away from broken — one `#[derive(Clone)]`, one `pub enum` — with no compiler warning; the defence is a greppable checklist and CI, and `unsafe` anywhere downstream voids it (hence `forbid(unsafe_code)` as policy).
- **Scale ceilings are real** (F-009, F-010, F-011). Type-level capacity costs `recursion_limit` $\approx N+3$ per crate and goes superlinear past $N \approx 500$; magnitudes are unusable as unary types; diagnostics erase aliases and spill long types to side files. A box of 100 bolts is fine; a warehouse is not a type.
- **One shape crashes the compiler** (F-034). Deterministically, with no diagnostic, at the mandated recursion limit. It is fenced by rule, not fixed.
- **The arithmetic burden is the modeller’s** (F-022, F-030, F-045). Every remainder and total is caller-stated; the compiler checks but never computes. Fallible processes roughly double the load.
- **Traceability is grep with a discipline** (F-021, F-037, F-038, F-053) — and two of its failure modes are *silently green*: a `Satisfies` tag inside a macro invocation is dropped without warning, and a reused requirement id across crates merges reports while the gate stays green. Both are documented with workarounds; neither is structurally solved.
- **Verification is not validation.** Tests prove the model self-consistent; nothing checks it against reality — CS-1’s energy figures are round-figure stand-ins, and every outcome-token constructor is a placeholder until calibrated (open question 1).
- **No tolerances, no time, no rates.** Quantities are exact integers — “1500 g $\pm$ 10 g” is inexpressible (open question 4); scheduling, duration and power are deliberately out of scope (R9, R15), so the History is a partial order, not a timeline.
- **The modeller must currently be Rust-literate.** The tutorials and the modeller’s guide lower the ramp; the DSL (§9.1) is the planned answer, and its make-or-break criterion is whether the curated error quality survives the extra layer.
- **Scope of evidence.** Everything above was measured on one toolchain (rustc 1.98.1, one platform) by one small team, on models of modest size. The case-study ladder exists to stress exactly this.

8 Related work

This project sits deliberately between several established fields; the debts are real even where the approach differs. No citations are given beyond names — the comparisons are general, and the repository makes no claim to a literature survey.

MBSE and SysML. Model-based systems engineering as practised — SysML and its successors, tools in the Capella tradition — gives systems a structured graphical language, with consistency checked by the modelling tool to the degree the tool implements it. Lavoisier differs in where the authority lives: the model *is* source code, and the checker is a production compiler maintained by a large community, with conservation, exclusivity and traceability as type-system theorems rather than tool-side validations. The cost is the loss of MBSE’s visual language and ecosystem — which steps 8–9 of the plan (generated diagrams and documents) exist to win back from the code side.

Typestate and session types. Encoding a value’s legal state transitions in its type — *typestate* — is a long-standing idea, and Rust’s ownership model made “*typestate pattern*” a common idiom (builders that change type as they are configured). R9’s one-type-per-processing-state rule and R12’s capacity-indexed suppliers are *typestate* applied to physical things; the project’s contribution is scale and discipline rather than novelty, plus the measured record of

where the diagnostics stop cooperating (F-009).

Linear and affine types. The “every resource used exactly once” ideal is linear typing, studied since Girard’s linear logic and implemented to varying degrees in research languages and in Haskell’s linear-types extension. Rust is affine (at most once), and F-002 is this paper’s quantification of exactly what that costs in practice — with the layered regime as an engineering answer where a linear language would have a theorem. A genuinely linear host language is the obvious alternative design; stable Rust was chosen for its maturity, diagnostics and reach.

Units-of-measure libraries. Checking dimensions in types is well-trodden: F# has units of measure in the language; Rust’s uom and C++’s mp-units do it in libraries. R7/R8 reimplement a small version deliberately (the experiment forbids dependencies, and the conservation asserts need the values, not just the dimensions); the per-currency dimensions of R19 are the same idea applied to money.

Formal methods. Specification languages with machine-checked consistency (the TLA+/Alloy/B tradition) prove far richer properties — temporal behaviour, refinement — at the cost of a separate artifact and specialist skills. Lavoisier checks a narrower class of property (balance, exclusivity, state, capacity) but keeps the model, the implementation of the checks, and the documentation in one artifact a working engineer can run with two commands.

9 Future work

9.1 The planned deliverables (PLAN steps 8–11)

- **Diagram generation** (step 8): three levels generated from the models — context (boundary crossings only), top-level (first-level processes), detailed (all levels) — GitHub-viewable with hyperlinks from diagram elements to code; committed SVG is the likely vehicle.
- **Process document generation** (step 9): candidate R20 made real — a generated, human-readable document per process, detailed enough for real-world implementation, built from what the conventions already provide: signatures as work-instruction skeletons, VALUE consts as the numbers, Placeholder: tags as the open-items list, the trace report as the compliance matrix, fill functions as the bill of materials.
- **A DSL** (step 10): a systems-engineering notation over the library, easier for humans to write and machines to read, with its own experiment batch (macro front-end vs external notation vs generation from specs). Error-message quality through the extra layer is the stated make-or-break criterion.
- **Model analysis** (step 11): a linter beyond ci.sh — unaccounted outputs, placeholder density, requirements with thin verification, convention violations, oversized processes — consolidated into one tool with a report.

9.2 The case-study ladder

Each remaining case study stresses something no earlier one did: CS-2 (bicycle puncture repair) exercises fallibility in anger — provisioned rework, purchasing, a qualified repairer; CS-3 (café order fulfilment) brings genuine concurrency and the first real per-branch History merge; CS-4 (a small-batch production run) pushes capacities toward the measured limits and demonstrates change impact — alter one resource type and catalogue what the compiler reports; CS-5 (two-site fulfilment with procurement) stretches the boundary — real suppliers replacing placeholders, two currencies with boundary exchange (R19’s only unexercised

clause), transport, and the model split across team-shaped crates. Each updates this paper.

9.3 Open questions

Four questions are logged at the end of `instructions.md` and remain open: **validation vs verification** (calibration data for every placeholder constant and outcome token — the model is consistent; is it true?); **document generation** (R20, step 9 above); **change impact and subsystem decomposition** (two apparent strengths — the compiler enumerating every consequence of a changed type, and crate boundaries as team interfaces — deserving systematic demonstration rather than anecdote); and **tolerances** (exact integers cannot say ± 10 g; min/max const pairs could, at real complexity cost — currently a logged limitation, not a plan).

10 Conclusions

The hypothesis survives, with measured edges. A stable, dependency-free type system can carry a working systems-engineering method: resources that cannot be duplicated, shared or conjured; processes whose books must balance in every dimension they touch, including both arms of failure; requirements that are defined, satisfied and verified by machinery that fails the build when the chain breaks; and a boundary where every assumption is a named, greppable placeholder. Two complete models pass a six-step gate, and the first specification-to-model round trip found and fixed a flaw in the verification tooling itself (F-053).

The edges are equally the result. The compiler's guarantees split cleanly in two: structural errors are caught everywhere, instantly; arithmetic errors are caught certainly but late, invisible to the editor. True linearity is out of reach — “nothing is silently lost” is a regime, not a theorem, and its two residual holes are documented. The ergonomic tax is real and quantified, from eight-const-parameter signatures to hand-maintained balances. And the project's most transferable lesson may be methodological rather than technical: agreeing requirements in writing, testing each risky claim in isolation before building, and logging every limitation with verbatim evidence produced a system whose weaknesses are as well-documented as its strengths — which is precisely what an engineering audience should demand of a method that proposes to audit *their* plans.

What remains — generated diagrams and work instructions, a humane notation, calibration against reality, and the rest of the case-study ladder — is laid out in §9, and future versions of this document will report on each as it lands.

Appendix A: Glossary

Term	Meaning
Resource	A modelled thing (material, item, person, tool, container), represented as a sealed type that cannot be cloned, copied or constructed outside the boundary
Process	A function standing for a work step: takes everything it needs by value, returns everything it produces (R1)
Sealed type	A struct with at least one private field and no public constructor, derives or data-to-Self functions (F-005)
Boundary	The model's edge: the only place resources are created (suppliers, fill/draw functions) or finally leave (consumers, sinks) (R12)
Supplier / Consumer	Boundary traits for discrete entry/exit, with capacity in the type; strict (nothing but self and the item) (R12)
Draw process	The continuous-resource counterpart: takes a container, returns the drawn amount plus the container at its caller-stated remainder (R15)
Placeholder	A <code>/// Placeholder</code> : tagged boundary object or constant standing for an undecided real-world counterpart; the model's named assumptions (R12)
Tripwire	A Drop implementation on a consumable resource that panics at test time if the value dies unconsumed (R1 layer 2, F-008)
Defuse	The sealed, sanctioned <code>mem: :forget</code> site a consumer uses to accept a tripwired resource (F-008, F-032)
Conservation assert	An inline <code>const { assert!(...) }</code> balancing a process's inputs and outputs per dimension; violations are E0080 at monomorphization (R3, F-001)
Post-monomorphization error	A compile error that fires during code generation, invisible to cargo editors and trybuild (F-001, F-003)
Kind trait	A trait bounding a catalogue type's parameter slot (<code>Size, Material</code>), making transposed arguments a construction-site error (R6, F-018)
Requirement trait	A named trait <code>ReqNNN...</code> whose doc comment starts with its <code>REQ-NNN</code> id, used as a process bound; the R10 pattern
<code>Satisfies: / Verifies: trace.sh</code>	Greppable doc tags linking types/processes and tests to requirement ids; <code>Satisfies:</code> is backed by a compile-checked assertion (R10, F-020)
<code>ci.sh</code>	The dependency-free shell script producing the traceability report; any warning fails CI (R10, EXP-06)
<code>test-support</code>	The six-step verification gate: full build, tests, clippy, plain production build, traceability, feature-placement audit (§4.4)
<code>test-support</code>	The cargo feature exposing downstream test fixtures; legal only under <code>[dev-dependencies]</code> (F-004)
Peano capacity	A type-level natural (<code>Succ<Succ<...<Zero>>></code>) counting a supplier/consumer's remaining capacity (R12, EXP-01)
One type per state	Each processing, failure and safety state is its own type, converted only by processes (R9, F-023)
Outcome token	A sealed, runtime-valued token injecting a fallible process's outcome at the boundary; flows cannot read it (R17, F-042)
Outcome bundle	A <code>#[must_use]</code> , deliberately Debug-less grouping returned per <code>Result</code> arm of a fallible process (R17, F-046/F-047)

Term	Meaning
History	The sealed boundary consumer recording consumed time and events: “the past” as a sink and the execution record, a partial order across branches (R16)
model-core	The reusable kernel library; downstream model crates (pilot-workshop, cs1-pot-of-tea) depend on it
F-NNN / R-NN / EXP-NN / REQ-NNN	A finding; a rule of the modelling approach; an experiment; a requirement of a modelled system

Appendix B: The rules R1–R19, one line each

Rule	One-line statement
R1	Processes conserve resources: everything in by value, everything out, no side effects; resources have private constructors; the layered regime covers silent loss
R2	Strict conservation: resources are never borrowed or shared; reusables are moved in and returned
R3	Splitting and combining amounts are processes, balance-checked by const asserts
R4	Compile errors expose model mismatches; two error classes (type-check vs post-monomorphization); CI never gates on cargo check alone
R5	Unit tests verify every transformation — every branch of every process arm
R6	Characteristics are types: parameterized catalogues with kind traits; requirements always as trait bounds; one-way bridging
R7	Quantities and units are types carrying integer values in base units (g, mm, ms, mm ² , mm ³ , mK, mA, J, minor currency units)
R8	The project writes its own units library; no external units/number crates
R9	The model describes connections, not sequences; one type per processing state; loose threading by default
R10	Requirements traceability: REQ-NNN traits, tag + compile-checked assertion, tagged tests, trace.sh as a CI gate
R11	A shared library of common types (Person, Organisation, Location, units)
R12	Suppliers and consumers at the boundary: strict, finite, type-level capacity, Peano-counted, on_unimplemented-messaged; placeholders allowed and marked
R13	Discrete items are separate objects, never counts; fixed-size collections wherever possible
R14	Findings and limitations are logged in FINDINGS.md with stable F-ids and evidence citations
R15	Continuous resources are drawn-down containers; amounts, not rates; waste is an ordinary conserved output; unbounded sources/sinks only at the boundary
R16	Execution history: consumed time goes to the sealed History consumer; per-branch histories merged at joins into a partial order

Rule	One-line statement
R17	Fallible processes return Result with both arms conserving; boundary outcome tokens; Debug-less bundles; rework bounded by provisioning
R18	Qualifications and safety as types: the Qualified wrapper, one type per safety state, REQ-phrased refusals
R19	Money as a conserved dimension: one dimension per currency, exact-price vendors, boundary-only integer-exact exchange

Appendix C: Findings index by theme

Theme	Findings
Post-monomorphization visibility	F-001, F-003, F-027, F-045
Silent loss and the layered regime	F-002, F-007, F-008, F-032, F-035, F-039, F-040
Privacy and sealing	F-005, F-006, F-026, F-031
Type-level numbers, capacity, diagnostics at scale	F-009, F-010, F-011, F-012, F-013
Boundary machinery	F-014, F-015, F-016, F-028, F-029, F-034
Characteristics, catalogues, states, flows	F-017, F-018, F-019, F-023, F-024, F-025
Quantity ergonomics	F-022, F-030, F-033
Traceability discipline	F-020, F-021, F-037, F-038, F-053
Kernel macros	F-036 (resolved), F-040
Execution history	F-041
Fallible processes	F-042, F-045, F-046, F-047, F-050
People, qualifications, safety	F-043, F-044, F-048, F-049, F-054
Money	F-051, F-052